Solution: Path With Minimum Effort

Problem Statement

Given a 2D array `heights[][]` of size `n x m`, where `heights[n][m]` represents the height of the cell `(n, m)`.

Find a path from the top-left corner to the bottom-right corner that minimizes the effort required to travel between consecutive points, where effort is defined as the absolute difference in height between two points. In a single step, you can either move `up`, `down`, `left` or `right`.

Determine the minimum effort required for any path from the first point to the last.

Example Generation

Example 1:

- Input: `[[1,2,3],[3,8,4],[5,3,5]]`
- Expected Output: `1`
- Justification: The path with the minimum effort is along the edges of the grid (right, right, down, down) which requires an effort of 1 between each pair of points.

Example 2:

- Input: `[[1,2,2],[3,3,3],[5,3,1]]`
- Expected Output: `2`
- Justification: The path that minimizes the maximum effort goes through (1,2,2,3,1), which has a maximum effort of 2 (from 3 to 1).

Example 3:

- Input: `[[1,1,1],[1,1,1],[1,1,1]]`
- Expected Output: `0`
- Justification: The path that minimizes the maximum effort goes through (1,1,1,1,1), which has a maximum effort of 0.

Solution

To solve this problem, we'll employ the Union Find algorithm. The core idea here is to sort all the edges (the differences in elevation between adjacent cells) in ascending order. Then, starting from the smallest edge, we'll progressively unite adjacent cells in the grid. The process of uniting cells involves linking together cells that can be reached with the current maximum effort level. This step is crucial because it helps to build a path incrementally while ensuring that we keep the effort as low as possible.

We continue this process of uniting cells until the top-left cell and the bottom-right cell are connected. At this point, the current maximum effort level represents the minimum effort needed to traverse from the start to the end. This method efficiently finds the path with the least resistance while considering all possible paths and their associated efforts.

The algorithm involves the following steps:

Step 1: Initialization

1. Create a `UnionFind` instance for a 3x3 grid, resulting in 9 elements (0 to 8).
2. Initialize an empty list to store edges.

Step 2: Building Edges

1. Iterate over each cell in the grid.
2. For each cell, calculate the absolute difference in elevation with its right and bottom neighbors (if they exist).

Step 3: Sorting Edges

1. Sort the edges by their elevation difference in ascending order.

Step 4: Perform Union-Find Operations

1. For each edge in the sorted list, perform the following:
   - Extract the two cells (cell1 and cell2) and their elevation difference from the edge.
   - Perform the `union` operation on cell1 and cell2.

Step 5: Union Operation

1. In the `union` function, find the roots of both cell1 and cell2 using the `find` method.
2. If the roots are different, join the two sets. If one set has a higher rank (depth), make it the parent of the other.
3. If both sets have equal rank, make one the parent of the other and increase the rank of the parent.

Step 6: Find Operation

1. In the `find` method, for a given cell, find the root of the set to which it belongs.
2. Implement path compression: Set each cell along the way to point directly to the root. This optimizes future `find` operations.

Step 7: Check for Connection

1. After each union operation, check if the start cell (0,0) and the end cell (2,2) are connected.
2. This is done by checking if the roots of cell 0 and cell 8 (in the UnionFind structure) are the same.
3. If they are connected, stop the process.

Step 8: Result

1. The elevation difference of the last processed edge that connected the start and end cells is the minimum effort required.

Algorithm Walkthrough

Step 1: Initialization

- Initialize a UnionFind instance for a 3x3 grid, resulting in 9 elements.
- Create an empty list to store the edges.

Step 2: Building Edges

- Iterate through each cell in the grid and calculate the absolute differences with its right and bottom neighbors (if they exist).
- For the input `[[1,2,3],[3,8,4],[5,3,5]]`, create edges with their differences:
  - Between (0,0) and (0,1): |1-2| = 1
  - Between (0,1) and (0,2): |2-3| = 1
  - Between (0,0) and (1,0): |1-3| = 2
  - Between (1,0) and (2,0): |3-5| = 2
  - Between (1,0) and (1,1): |3-8| = 5
  - Between (1,1) and (1,2): |8-4| = 4
  - Between (0,1) and (1,1): |2-8| = 6
  - Between (1,1) and (2,1): |8-3| = 5
  - Between (0,2) and (1,2): |3-4| = 1
  - Between (1,2) and (2,2): |4-5| = 1
  - Between (2,0) and (2,1): |5-3| = 2
  - Between (2,1) and (2,2): |3-5| = 2

Step 3: Union-Find Operations

First, we sort the edges by their differences:

1. Edges with Difference = 1:
   - (0,0) and (0,1): Difference = 1
   - (0,1) and (0,2): Difference = 1
   - (0,2) and (1,2): Difference = 1
   - (1,2) and (2,2): Difference = 1
2. Edges with Difference = 2:
   - (0,0) and (1,0): Difference = 2
   - (1,0) and (2,0): Difference = 2
   - (2,0) and (2,1): Difference = 2
   - (2,1) and (2,2): Difference = 2
3. Edges with Higher Differences (not initially relevant for the desired path).

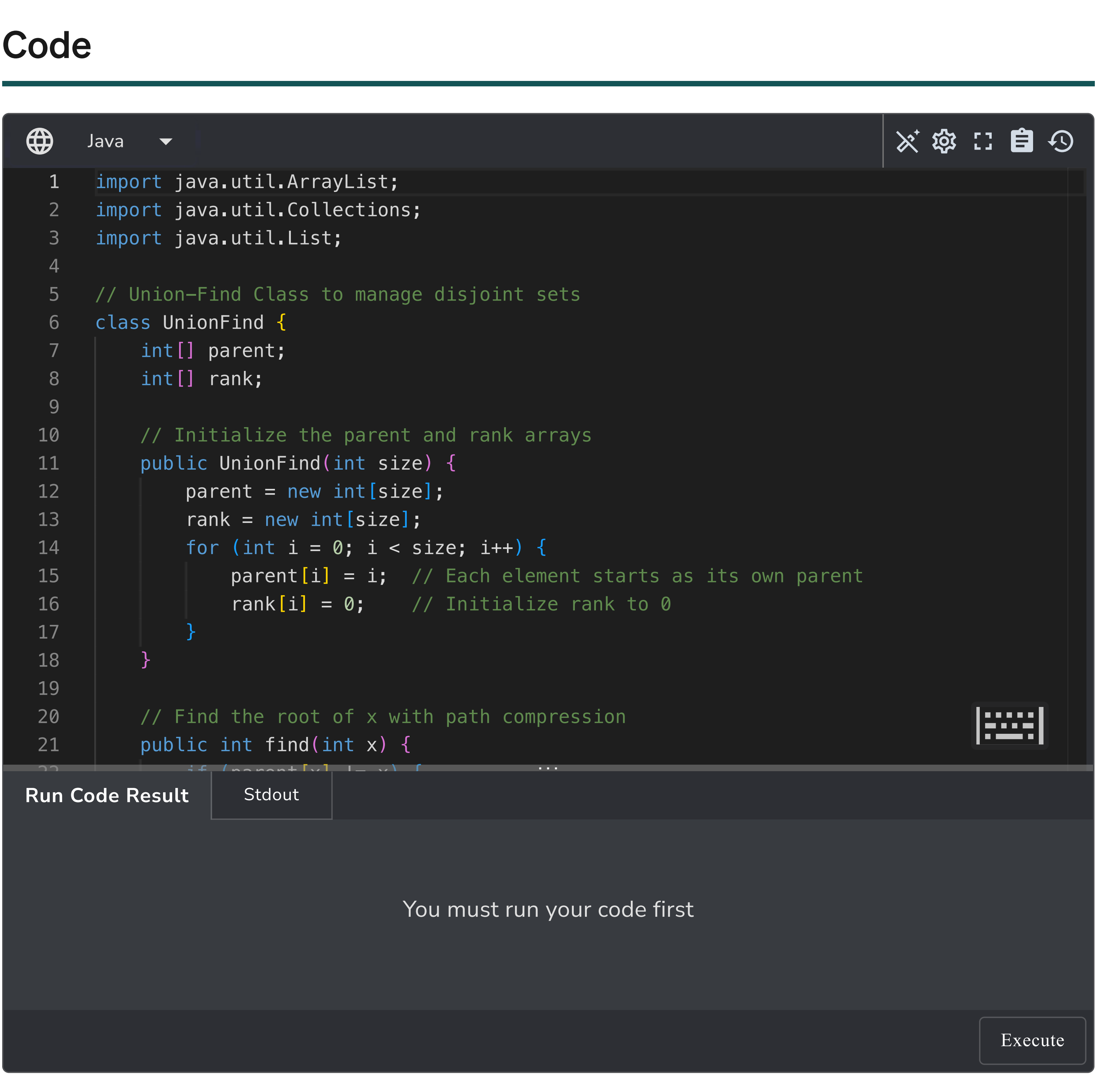
Now, let's perform the Union-Find operations:

Processing Edges with Difference = 1

1. **Union (0,0) and (0,1)**:
   - Connects cells 0 and 1. Parents: `[0, 0, 2, 3, 4, 5, 6, 7, 8]`.
2. **Union (0,1) and (0,2)**:
   - Connects cells 1 and 2 (and hence 0 with 2). Parents: `[0, 0, 0, 3, 4, 5, 6, 7, 8]`.
3. **Union (0,2) and (1,2)**:
   - Connects cells 2 and 5. Parents: `[0, 0, 0, 3, 4, 0, 6, 7, 8]`.
4. **Union (1,2) and (2,2)**:
   - Connects cells 5 and 8. Parents: `[0, 0, 0, 3, 4, 0, 6, 7, 0]`.
   - At this step, cells 0 and 8 are connected.

- The last edge difference is `1`.

Code



```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

// Union-Find Class to manage disjoint sets
class UnionFind {
    int[] parent;
    int[] rank;

    // Initialize the parent and rank arrays
    public UnionFind(int size) {
        parent = new int[size];
        rank = new int[size];
        for (int i = 0; i < size; i++) {
            parent[i] = i;  // Each element starts as its own parent
            rank[i] = 0;    // Initialize rank to 0
        }
    }

    // Find the root of x with path compression
    public int find(int x) {
```

You must run your code first

Complexity Analysis

Time Complexity:

- The time complexity of this algorithm is primarily dictated by the sorting of the edges.
- If there are `n` cells, we can have at most `2n` edges (since each cell can have at most 2 outgoing edges to its right and bottom neighbors).
- Sorting `2n` edges takes `O(2n log(2n))`, which simplifies to `O(n log n)`.
- The union-find operations (find and union) take `O(n)` time each.
- Hence, the overall time complexity is `O(n log n)`.

Space Complexity:

- The space complexity is `O(n)` for storing the parents and ranks in the union-find data structure and the edge list.